
A TITLE

William Chang

Department of Applied Mathematics
University of California, Los Angeles
Los Angeles, CA
<https://williamc.me/>

ABSTRACT

Automated environment generators—increasingly, large language models that write environment programs—train reinforcement-learning agents on curricula of progressively harder tasks, but waste compute by training a policy on each candidate environment before its usefulness is known. We ask whether the value a policy will attain on a new environment can be *predicted* from cheap, pre-deployment descriptors, so that expensive training is reserved for useful environments. We decompose the value gap between two parameterized environments into a reward-feature term, handled by successor features, and a dynamics term that we relate to a small set of *geometric* descriptors read directly from the environment specification. The central hypothesis—that policy occupancy is approximately Lipschitz in these features—is tested on three procedurally generated domains, and geometry predicts difficulty in proportion to how geometric the difficulty is: strongly in shaped MiniGrid LAVAGAP ($\rho \approx 0.7$), partially in Sokoban ($\rho \approx 0.35$), and not at all in semantic Karel ($\rho \approx 0$), where a cheap dynamics-aware probe recovers the missing signal. We then put the predictor to work as a downstream curriculum tool. As a prefilter it matches full-pool generalization at up to $15\times$ less compute by exploiting pool redundancy. As a curriculum signal in a hard-exploration LAVACROSSING regime where a flat schedule never succeeds, the cheap, oracle-free geometric difficulty orders tasks well enough to match an *expensive* empirical oracle—reaching the same held-out success at a fraction of the difficulty-estimation cost. No component of the method uses a language model.

1 Introduction

A growing line of work uses automated environment generators — including large language models writing environment programs — to train reinforcement learning agents on curricula of progressively challenging tasks [???]. These systems share a common loop: propose an environment, train a policy on it, evaluate, repeat. The loop is empirically effective but computationally wasteful, because every proposed environment consumes substantial compute before its usefulness for the current policy is known.

A natural question is whether one can *predict* the value attained by a policy on a candidate environment from cheap, pre-deployment descriptors of the environment. If so, expensive policy training can be reserved for environments whose predicted value lies in a useful range, and the LLM-driven proposer can be used as a generator whose outputs are filtered by a predictive model.

For the special case where environments share dynamics and differ only in reward, successor features and generalized policy improvement (SF&GPI) [??] provide such a predictor with explicit performance bounds. The bound depends on the distance between reward-feature vectors and is tight in the limit of shared dynamics. In the more general case relevant to robotics — where dynamics also vary across environments, often through changes in geometry or contact structure — no analogous predictor is established. Existing curriculum methods relying on rollout-derived signals (TD error in PLR [?], regret in PAIRED and ACCEL [??], transition-prediction error in TRACED [?]) all require executing the policy in the candidate environment, which is the cost we wish to avoid.

This paper proposes a framework for predicting policy value on new environments by combining (i) the existing SF-based reward-transfer machinery and (ii) a *geometric task descriptor* extracted directly from the environment’s specification. The framework rests on a single working hypothesis: that for parameterized environment families, the policy-induced occupancy is approximately Lipschitz in a small set of geometric features that can be read from the environment program. This hypothesis is empirically testable on standard procedurally generated benchmarks, and its truth or falsity is the main scientific question we propose to investigate.

Contributions. (1) A decomposition of the value gap between two parameterized environments into reward-feature residual and dynamics-feature residual, generalizing the SF&GPI prediction setting. (2) A predictive model that combines the two terms via a learned regressor on observed (environment, policy, return) triples. (3) A curriculum criterion based on predicted difficulty and descriptor-space coverage, intended for use as a prefilter in front of LLM-driven environment generators. (4) An empirical demonstration, in a hard-exploration LAVACROSSING regime where a flat schedule never succeeds, that the cheap oracle-free geometric difficulty signal—used to *order* tasks from easy to hard—drives a neural RL curriculum to the same held-out performance as an expensive empirical oracle, at negligible difficulty-estimation cost.

2 Preliminaries

2.1 Parameterized Environments

Let Θ be a space of environment parameters. In practice Θ may be the set of valid outputs of an environment-generating program (an LLM-produced Python function, a procedural generator, or a hand-coded parameterization). Each $\theta \in \Theta$ specifies a POMDP

$$\mathcal{M}_\theta = (\mathcal{S}, \mathcal{A}, \mathcal{O}, T_\theta, R_\theta, I, \gamma), \quad (1)$$

in which both the transition kernel $T_\theta : \mathcal{S} \times \mathcal{A} \rightarrow \Delta(\mathcal{S})$ and the reward function $R_\theta : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ depend on θ . A stationary policy is a map $\pi : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ and its value is

$$V^\pi(\theta) = \mathbb{E}_{\pi, T_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R_\theta(s_t, a_t) \mid s_0 \sim I \right]. \quad (2)$$

We assume access to:

- A set of training environments $\mathcal{T}_{\text{train}} = \{\theta_1, \dots, \theta_n\} \subset \Theta$.
- A library of policies $\Pi = \{\pi_1, \dots, \pi_m\}$ obtained from prior training iterations, each with at least one associated return measurement $V^{\pi_j}(\theta_i)$ on some $\theta_i \in \mathcal{T}_{\text{train}}$.
- A candidate *new* environment $\theta_{\text{new}} \in \Theta$ proposed by some generator.

The prediction problem is: given θ_{new} , $\mathcal{T}_{\text{train}}$, Π , and observed returns, estimate $V^\pi(\theta_{\text{new}})$ for a chosen policy π *without* rolling π in $\mathcal{M}_{\theta_{\text{new}}}$.

2.2 Reward-Only Transfer via Successor Features

The classical SF&GPI setting assumes shared dynamics ($T_\theta \equiv T$) and a linear reward decomposition

$$R_\theta(s, a) = \phi(s, a)^\top w_\theta, \quad \phi : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}^d, \quad w_\theta \in \mathbb{R}^d. \quad (3)$$

For any policy π , the *successor features* are

$$\psi^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t, a_t) \mid s_0 = s, a_0 = a \right] \in \mathbb{R}^d, \quad (4)$$

and the action-value factors as $Q_\theta^\pi(s, a) = \psi^\pi(s, a)^\top w_\theta$. ? show that if π_i is optimal for task w_i and we apply the GPI policy $\pi_{\text{GPI}}(s) = \arg \max_a \max_i \psi^{\pi_i}(s, a)^\top w_\theta$ to a new task w_θ , then

$$Q_\theta^*(s, a) - Q_\theta^{\pi_{\text{GPI}}}(s, a) \leq \frac{2}{1 - \gamma} \min_i \|w_\theta - w_i\|. \quad (5)$$

This is a pre-deployment prediction: the optimality gap is bounded by a reward-vector distance computable from θ_{new} alone. The bound applies only when dynamics are shared.

2.3 The Dynamics Gap

When dynamics also vary, the successor features themselves depend on θ :

$$\psi_\theta^\pi(s, a) = \mathbb{E}_{\pi, T_\theta} \left[\sum_{t=0}^{\infty} \gamma^t \phi(s_t, a_t) \mid s_0 = s, a_0 = a \right]. \quad (6)$$

The value difference between two environments θ, θ' admits the decomposition

$$\begin{aligned} V^\pi(\theta) - V^\pi(\theta') &= \mathbb{E}_{s \sim I} \left[\psi_\theta^\pi(s)^\top w_\theta - \psi_{\theta'}^\pi(s)^\top w_{\theta'} \right] \\ &= \underbrace{\mathbb{E}_{s \sim I} [\psi_{\theta'}^\pi(s)^\top (w_\theta - w_{\theta'})]}_{\text{(A) reward-feature gap}} + \underbrace{\mathbb{E}_{s \sim I} [(\psi_\theta^\pi(s) - \psi_{\theta'}^\pi(s))^\top w_\theta]}_{\text{(B) dynamics gap}}. \end{aligned} \quad (7)$$

Term (A) is controlled by reward-vector distance, exactly as in (5). Term (B) reflects the change in the policy’s discounted feature occupancy under different dynamics, and has no closed-form bound in terms of θ in general. The simulation lemma [?] gives a bound in terms of total variation distance between T_θ and $T_{\theta'}$, but TV distance is not directly computable from the environment programs.

2.4 Geometric Descriptors via Kinodynamic Reachability Graphs

We bound or estimate term (B) of (7) using a structured descriptor built from the environment program in three layers of increasing fidelity. Each layer adds information without rolling out any policy.

Layer 1: Local geometric features (g_{loc}). The simplest descriptor reads scalars directly from the environment program. For terrain-based environments these include maximum gap width and step height along the optimal-direction profile, slope statistics (mean and maximum gradient), height-field roughness (standard deviation along the path), and goal-distance statistics. These features capture whether the robot’s hard kinodynamic limits — maximum jump distance, maximum climb height, slope tolerance — are exceeded *locally* along an idealized trajectory. We denote this layer

$$g_{\text{loc}}(\theta) \in \mathbb{R}^{k_1}. \quad (8)$$

Layer 2: Kinodynamic reachability graph (g_{reach}). Local features cannot distinguish a terrain where the path is feasible from one where it is not, because they ignore global structure. To capture global feasibility we construct the *kinodynamic feasibility graph*.

Definition 1 (Kinodynamic feasibility graph) Let $\mathcal{V}_\theta \subset \mathcal{S}$ be a discretization of state space induced by θ (e.g., a grid over the terrain). The kinodynamic feasibility graph

$$G_\theta = (\mathcal{V}_\theta, \mathcal{E}_\theta), \quad (u, v) \in \mathcal{E}_\theta \iff \begin{array}{l} \text{a one-step transition from } u \text{ to } v \\ \text{satisfies the robot's mechanical capability} \\ \text{limits under environment } \theta. \end{array} \quad (9)$$

We write $A_\theta \in \{0, 1\}^{|\mathcal{V}_\theta| \times |\mathcal{V}_\theta|}$ for its adjacency matrix and D_θ for its degree matrix.

Combinatorial properties of G_θ give the second descriptor layer:

$$g_{\text{reach}}(\theta) = [\text{diam}(G_\theta), \text{mincut}(s_0, G_\theta), \#\{\text{bottleneck edges}\}, \#\{\text{components}\}, \dots] \in \mathbb{R}^{k_2}. \quad (10)$$

The minimum cut between the spawn region and the goal region is the classical *narrow-passage* quantity from motion planning. Layer 2 is feasible to compute in $O(|\mathcal{V}_\theta|^2)$ time and is what classical curriculum methods would call “solvability.”

Layer 3: Spectral features (g_{spec}). Combinatorial features summarize G_θ but ignore its global geometry. The normalized Laplacian

$$L_\theta = I - D_\theta^{-1/2} A_\theta D_\theta^{-1/2} \quad (11)$$

has eigenvalues $0 = \lambda_1(L_\theta) \leq \lambda_2(L_\theta) \leq \dots \leq 2$. The Cheeger inequality [?] relates λ_2 to the graph’s bottleneck structure:

$$\frac{\lambda_2(L_\theta)}{2} \leq h(G_\theta) \leq \sqrt{2\lambda_2(L_\theta)}, \quad (12)$$

where $h(G_\theta) = \min_S |\partial S| / \min(\text{vol}(S), \text{vol}(\bar{S}))$ is the Cheeger constant. Small λ_2 certifies the existence of a narrow bottleneck. Spectral features form the third layer:

$$g_{\text{spec}}(\theta) = [\lambda_2(L_\theta), \dots, \lambda_{k_3+1}(L_\theta)] \in \mathbb{R}^{k_3}. \quad (13)$$

Combined descriptor. The full descriptor is the concatenation

$$g(\theta) = [g_{\text{loc}}(\theta); g_{\text{reach}}(\theta); g_{\text{spec}}(\theta)] \in \mathbb{R}^{k_1+k_2+k_3}. \quad (14)$$

The layers are complementary: g_{loc} catches local capability violations that need not affect global graph structure (a single impossible jump in an otherwise easy terrain); g_{reach} catches feasibility at the path level (is there *any* feasible trajectory?); g_{spec} catches geometry of the feasible trajectory space and its bottlenecks (how narrow is the corridor of successful trajectories?).

2.5 Connecting Geometric Features to the Dynamics Gap

We now justify why the descriptor (14) is the right quantity to control term (B) of (7). The argument is standard linear algebra applied to the SF Bellman equation.

The successor feature for policy π under dynamics T_θ satisfies

$$\psi_\theta^\pi = \phi + \gamma T_\theta^\pi \psi_\theta^\pi, \quad \psi_\theta^\pi = (I - \gamma T_\theta^\pi)^{-1} \phi, \quad (15)$$

where T_θ^π is the policy-induced transition operator under θ . The resolvent identity gives, for any two environments θ, θ' ,

$$\psi_\theta^\pi - \psi_{\theta'}^\pi = \gamma (I - \gamma T_\theta^\pi)^{-1} (T_\theta^\pi - T_{\theta'}^\pi) (I - \gamma T_{\theta'}^\pi)^{-1} \phi, \quad (16)$$

and therefore the operator-norm bound

$$\|\psi_\theta^\pi - \psi_{\theta'}^\pi\| \leq \frac{\gamma}{(1-\gamma)^2} \|T_\theta^\pi - T_{\theta'}^\pi\|_{\text{op}} \|\phi\|. \quad (17)$$

Inequality (17) is rigorous and standard. It reduces the dynamics gap (B) to controlling the operator-norm distance between transition kernels under π .

Adjacency control. For any policy π that assigns zero probability to kinodynamically infeasible transitions — a property satisfied by any policy that has learned to avoid catastrophic failures — the policy-induced transition operator T_θ^π is supported on $\mathcal{E}_\theta$. A direct argument gives:

Proposition 2 (Adjacency control of the transition operator) *Let π be a stochastic policy supported on kinodynamically feasible actions for both θ and θ' , and assume $\mathcal{V}_\theta = \mathcal{V}_{\theta'} = \mathcal{V}$. Then*

$$\|T_\theta^\pi - T_{\theta'}^\pi\|_{\text{op}} \leq C_\pi \cdot \|A_\theta - A_{\theta'}\|_F, \quad (18)$$

where C_π depends on $\|\pi\|_\infty$ and on the maximum degree of $G_\theta \cup G_{\theta'}$.

The proof is direct: $T_\theta^\pi(u, v) - T_{\theta'}^\pi(u, v)$ is nonzero only at indices (u, v) where the feasibility pattern differs between θ and θ' . Summing over such indices gives the Frobenius adjacency distance up to row-mass constants. The shared-discretization assumption is met for procedurally generated grid environments; for continuous-control terrains it can be approximated by a common reference grid.

Combining (17) and Proposition 2,

$$\|\psi_\theta^\pi - \psi_{\theta'}^\pi\| \leq \frac{\gamma C_\pi \|\phi\|}{(1-\gamma)^2} \cdot \|A_\theta - A_{\theta'}\|_F. \quad (19)$$

Role of spectral features. The bound (19) uses $\|A_\theta - A_{\theta'}\|_F$, not Laplacian eigenvalues. Spectral features enter for two reasons:

1. **Compression.** For large $|\mathcal{V}|$ the full adjacency is high-dimensional and hard to feed to a learned regressor. The leading Laplacian eigenvalues $\{\lambda_2, \dots, \lambda_{k_3+1}\}$ are a low-dimensional summary that, by Weyl's inequality, is Lipschitz in $\|A_\theta - A_{\theta'}\|$:

$$|\lambda_i(L_\theta) - \lambda_i(L_{\theta'})| \leq \|L_\theta - L_{\theta'}\|_{\text{op}}. \quad (20)$$

Hence g_{spec} inherits the consistency of A_θ as a difficulty descriptor.

2. **Interpretability.** The spectral gap and Cheeger constant identify whether a bottleneck exists in G_θ — a single scalar that often determines whether a curriculum step is at the right difficulty.

Hypothesis 3 (Geometric Lipschitz, refined) *For each policy π in the policy library Π there exist constants $L_\pi \geq 0$ and $\epsilon_\pi \geq 0$ such that for all $\theta, \theta' \in \Theta$,*

$$\|\mathbb{E}_{s \sim I}[\psi_\theta^\pi(s) - \psi_{\theta'}^\pi(s)]\|_2 \leq L_\pi \cdot d_g(\theta, \theta') + \epsilon_\pi, \quad (21)$$

where d_g is a learnable metric on the descriptor space (14).

Inequality (19) gives a partial proof of Hypothesis 3: it establishes the bound for the A_θ -Frobenius component of d_g under the feasibility-respecting policy assumption, with explicit $L_\pi = \gamma C_\pi \|\phi\| / (1 - \gamma)^2$ and $\epsilon_\pi = 0$. The full hypothesis, including the local and spectral layers and arbitrary stochastic policies, remains the empirical claim we propose to test.

Remark 4 *Substituting (21) into (7) and using (5),*

$$|V^\pi(\theta) - V^\pi(\theta')| \leq \underbrace{\|\psi_{\theta'}^\pi\|_\infty \cdot \|w_\theta - w_{\theta'}\|}_{\text{reward term, from SF\&GPI}} + \underbrace{\|w_\theta\| \cdot (L_\pi d_g(\theta, \theta') + \epsilon_\pi)}_{\text{dynamics term, from kinodynamic graph}}. \quad (22)$$

The reward term uses descriptor w_θ and is the classical SF&GPI bound [?]. The dynamics term uses descriptor $g(\theta)$ from (14) and is the new contribution.

3 Framework: Predictive Difficulty

3.1 Task Descriptor

We concatenate reward-feature and geometric descriptors:

$$z(\theta) = [w_\theta; g(\theta)] \in \mathbb{R}^{d+k}. \quad (23)$$

A policy is summarized by an embedding $e_\pi \in \mathbb{R}^m$, which may be derived from network activations, recent reward-component statistics on a probe set of environments, or both.

3.2 Value Predictor

We learn a regressor

$$\hat{V}(\theta, \pi) = f_\phi(z(\theta), e_\pi) \quad (24)$$

trained on observed triples $\{(\theta_i, \pi_j, V^{\pi_j}(\theta_i))\}$ collected during prior curriculum iterations. The loss is mean squared error,

$$\mathcal{L}(\phi) = \sum_{i,j} (f_\phi(z(\theta_i), e_{\pi_j}) - V^{\pi_j}(\theta_i))^2. \quad (25)$$

Hypothesis 3 implies that f_ϕ should be learnable with sample complexity polynomial in $d + k + m$ when the function class is appropriately regularized.

3.3 Predicted Difficulty

The predicted difficulty of environment θ for policy π is

$$\hat{D}(\theta, \pi) = \hat{V}^*(\theta) - \hat{V}(\theta, \pi), \quad \hat{V}^*(\theta) = \max_{\pi_k \in \Pi} \hat{V}(\theta, \pi_k). \quad (26)$$

We use the policy library Π as a stand-in for the unobserved $V^*(\theta)$, in the spirit of GPI.

3.4 Curriculum Selection

The curriculum selects the next environment by maximizing a utility combining learnability and coverage:

$$\theta_{\text{next}} = \arg \max_{\theta \in \Theta_{\text{cand}}} \mathcal{U}(\theta; \pi), \quad (27)$$

where

$$\mathcal{U}(\theta; \pi) = \alpha \cdot \mathbb{I}[\hat{D}(\theta, \pi) \in [D_{\min}, D_{\max}]] + \beta \cdot \text{Cov}(\theta; \mathcal{T}_{\text{train}}). \quad (28)$$

The first term enforces a zone of proximal development [?] — neither too easy nor too hard — as identified in prior learnability-based curricula [?]. The second is a ridge-leverage coverage score in descriptor space:

$$\text{Cov}(\theta; \mathcal{T}_{\text{train}}) = z(\theta)^\top (ZZ^\top + \lambda I)^{-1} z(\theta), \quad Z = [z(\theta_1) \cdots z(\theta_n)]. \quad (29)$$

3.5 Use as a Prefilter

In an LLM-driven curriculum system, the generator proposes a batch $\{\theta_1, \dots, \theta_N\}$. We evaluate $\mathcal{U}(\theta_i; \pi)$ on each, rank, and train only on the top- k with $k \ll N$. Predictor accuracy translates directly into a multiplicative reduction in policy-training compute.

Empirical questions. The framework is meaningful only insofar as the following hold:

1. Hypothesis 3: geometric descriptors $g(\theta)$ admit a Lipschitz-like control of the dynamics gap (B) in (7) for the environment family of interest.
2. The regressor f_ϕ generalizes to held-out environments with error small enough to be useful for ranking.
3. The prefilter yields a net compute saving relative to the unfiltered baseline at matched final policy performance.

These are the testable claims of the framework, and they are the targets of the experimental section to follow. The bulk of the paper concerns the *first* of these claims — whether a cheap, pre-deployment descriptor can predict realized task difficulty — which is logically prior to the curriculum and compute-saving claims and which we test directly in three procedurally generated program-synthesis domains. Having established it, we then test the *third* claim — that the predictor yields a net compute saving as a prefilter — as a downstream use case in Section 6.6.

4 Instantiating the Difficulty Predictor

The framework of Section 3 is agnostic to the choice of solver: $V^\pi(\theta)$ is the value attained by *some* agent π on environment θ , and difficulty is the gap between the best attainable value and the value a given agent reaches. To test Hypothesis 3 concretely we must (i) fix a reference agent so that “difficulty” becomes a measurable number, (ii) verify that this number is a stable property of the environment rather than an artifact of the agent’s randomness, and (iii) specify the cheap descriptors whose predictive power we evaluate. We instantiate all three in a *programmatic policy* setting, where the agent searches over programs in a domain-specific language (DSL) rather than over neural-network weights. This choice is deliberate: programmatic policies make the environment’s combinatorial and geometric structure the dominant determinant of difficulty, which is precisely the regime in which Hypothesis 3 is most sharply falsifiable.

4.1 Reference Solver and the Difficulty Label

We take as reference solver a stochastic local search over programmatic policies, namely Hill Climbing (HC) over a DSL program space [??]. Starting from a randomly initialized program, HC repeatedly proposes mutations, evaluates each candidate in the environment, and retains improving candidates, for a fixed budget of T program evaluations. Let $r_{\text{HC}}^*(\theta; \xi) \in [0, 1]$ denote the best (task-normalized) reward reached by an HC run on environment θ with search-randomness seed ξ . The per-run difficulty is $1 - r_{\text{HC}}^*(\theta; \xi)$: an environment on which HC easily discovers a high-reward program is easy; one on which the search stalls at low reward is hard.

A single HC run conflates two sources of variation: the genuine hardness of θ , and the luck of the particular search trajectory ξ (random initialization and mutation order). We therefore define the difficulty *label* as the expectation over search randomness, estimated by averaging R independent runs with distinct seeds:

$$D_{\text{HC}}(\theta) = 1 - \frac{1}{R} \sum_{r=1}^R r_{\text{HC}}^*(\theta; \xi_r). \quad (30)$$

$D_{\text{HC}}(\theta)$ is an empirical estimate of the realized difficulty $\hat{D}(\theta, \pi_{\text{HC}})$ of Section 3, with the policy library collapsed to the single reference solver π_{HC} . We use this same DSL+HC solver uniformly across all three domains of Section 6, including Sokoban, for which we define a small push/move DSL (Section 6.5). The definition (30) is nonetheless agnostic to the search procedure: where an environment also admits a natural non-programmatic search formulation — as Sokoban does over board states — we use it as an independent cross-check and confirm the label is invariant to the choice (Section 6.5, Figure 1).

4.2 Reliability of the Difficulty Label

Before asking whether *any* descriptor predicts D_{HC} , we must establish that D_{HC} measures a repeatable property of θ at all. If re-running the entire measurement with fresh search seeds reorders the environments, then there is no stable target to predict and a low predictor score is uninformative. We quantify this with a split-half reliability. Given the R

per-environment runs, we randomly partition the seeds into two halves, recompute the averaged label (30) within each half, and measure the rank agreement between the two halves by Spearman correlation; we apply the Spearman–Brown correction to estimate the reliability of the full R -run label and average over 20 random partitions:

$$\rho_{\text{rel}} = \mathbb{E}_{\text{splits}} \left[\frac{2 \rho_s(D^{(A)}, D^{(B)})}{1 + \rho_s(D^{(A)}, D^{(B)})} \right], \quad (31)$$

where $D^{(A)}, D^{(B)}$ are the half-labels and ρ_s is Spearman’s rank correlation. A high ρ_{rel} certifies that difficulty is a real, agent-independent property of the environment; only on environment families that clear this bar is the predictor question well posed. Reliability is distinct from predictive power: the former compares the label to an independent replicate of *itself*, the latter compares a descriptor-based prediction to the label.

4.3 Predictor 1: Geometric Descriptor Regressor

The primary predictor is the geometric descriptor $g(\theta)$ of Section 2.4, instantiated for grid environments. From the environment program alone, with no rollouts, we build the kinodynamic feasibility graph G_θ over free cells and extract the three descriptor layers: local features g_{loc} (grid dimensions, free/wall counts, dead-ends, corridor and branch statistics, goal-distance statistics), reachability features g_{reach} (graph diameter, connected components, degree statistics, articulation points and bridge counts, start-region reachability), and spectral features g_{spec} (the leading eigenvalues of the normalized Laplacian L_θ). For reward-bearing tasks we additionally include a small set of reward-structure features (e.g. number and placement of target markers). The concatenated descriptor $g(\theta)$ is mapped to a scalar difficulty estimate by a learned regressor; we use a Random Forest, which is nonparametric, requires no feature scaling, and is robust on the modest sample sizes available per task. We emphasize that this predictor uses *no* language model: the only learned component is the Random Forest trained on (descriptor, label) pairs.

4.4 Predictor 2: Cheap Dynamics-Aware Probe

The geometric predictor is purely static: it never executes the environment. As a complementary, *dynamics-aware* predictor that is still far cheaper than solving the task, we introduce a cheap rollout probe. The probe executes a small, fixed library of short programs in the environment — e.g. “move forward”, “move and place a marker”, “follow a wall while clear”, together with a handful of randomly sampled DSL programs — and records the reward, crash flag, and steps taken by each. These cheap rollouts sense the *interaction* between the grid layout and the reward function, which the static descriptor cannot observe. We use the probe rewards (per-program rewards and random-program reward statistics) as features for a difficulty regressor, and we also report the single most predictive probe feature on its own.

A natural truncated-search variant of this probe replaces the fixed program library with a severely truncated run of the reference solver itself: starting from a *random* base program r_0 (no language model) and evaluating a handful of perturbations under a budget $T_{\text{probe}} \ll T$ (on the order of ten evaluations rather than thousands). Its difficulty score is the complement of the best reward reached within the budget,

$$D_{\text{probe}}(\theta) = 1 - r_{T_{\text{probe}}}^*(\theta), \quad (32)$$

mirroring the reference label $D_{\text{HC}} = 1 - \max_t r_t$ (Eq. 30). The intuition is identical: on an easy environment a few perturbations quickly reach high reward (the local search landscape is informative), whereas on a hard environment the reward stays stagnant under the same effort. We evaluate both the fixed-library probe and this truncated-search variant in Section 6.4.

Does the probe defeat the purpose? A natural objection is that running *any* rollout in the environment contradicts the goal of avoiding environment execution. The distinction is one of scale, and it is the crux of the predictor’s value proposition. The reference solver expends T evaluations (thousands) to *find* a programmatic solution; the probe expends a handful of cheap rollouts (well under a second, roughly an order of magnitude cheaper on easy grids and substantially more on hard ones) to *classify* the environment as easy or hard without solving it. The probe trades the static predictor’s zero-rollout property for a small, bounded number of rollouts in exchange for a dynamics-aware signal; the geometric predictor remains the fully execution-free option. We report both so that the static and cheap-dynamic regimes can be compared directly.

4.5 Reward Shaping for Sparse-Reward Environments

The difficulty label (30) is only meaningful when the reference solver produces a graded signal. In environments whose native reward is sparse — a single positive reward on exact task completion and zero elsewhere — the HC search landscape is flat: almost every program scores zero, no mutation is improving, and the search terminates immediately

on *every* environment, collapsing D_{HC} to its maximum with zero variance. We confirm this degeneracy is not a budget artifact (Section 6.2). To recover a graded label we apply a task-appropriate dense shaping that awards partial credit for progress toward the goal; for navigation environments we use a shortest-path distance-to-goal potential,

$$r_{\text{shaped}}(\theta) = \frac{d_{\text{start}} - d_{\text{min}}}{d_{\text{start}}}, \quad (33)$$

where d_{start} is the goal distance from the start cell and d_{min} the closest distance the agent reaches, with $r_{\text{shaped}} = 1$ on goal arrival. Shaping changes the reward but not the environment geometry, and it is what makes the descriptor question testable in the sparse regime.

5 Experimental Setup

5.1 Domains and Tasks

We evaluate in three procedurally generated program-synthesis domains, chosen to span a spectrum from purely semantic to purely geometric difficulty: Karel (semantic), MiniGrid (geometric), and Sokoban (mixed). For completeness our screen covers *every* task available in the two grid-world domains — all 12 Karel tasks and all 3 MiniGrid tasks — so that the geometric/semantic characterization is established over the full task suite rather than a hand-picked subset; tasks that fail the reliability gate are reported as such rather than omitted.

Karel. The Karel domain [?] places an agent on a grid with walls and markers; tasks are specified by a per-task reward function. We study tasks whose objective is fixed but whose grid layout varies with a generation seed. The headline tasks are TOPOFF (place a marker on each pre-placed target marker along a row) and SNAKE (grow a snake by consuming markers). To test whether the Karel conclusion generalizes beyond these two, we additionally screen eight further tasks at the full protocol — DOORKEY, MAZE, WALLAVOIDER, ONESTROKE, PATHFOLLOW, STAIRCLIMBER, CLEANHOUSE, FOURCORNERS — alongside the earlier-screened HARVESTER and SEEDER (the latter two reported below as degenerate because their difficulty exhibits negligible across-grid variance). As detailed in Section 6.1, most of these tasks either fall below the reliability bar or reproduce the geometry-blind pattern, with WALLAVOIDER — whose hardness is itself spatial — the one geometric exception.

MiniGrid. The MiniGrid domain provides spatial navigation and manipulation tasks. Our primary geometric task is LAVAGAP: a strip of lava spans the room with a single passable gap, and the agent must reach a goal cell on the far side; stepping into lava is fatal. The difficulty of LAVAGAP is intrinsically geometric — governed by the gap’s location and the induced detour — making it the natural positive control for the descriptor hypothesis, and it is the task we carry through the quantitative study (and the downstream curriculum experiment of Section 6.6). The domain also contains REDBLUEDOOR (open the red then blue door in order) and PUTNEAR (place an object beside a target); these are sequencing/manipulation tasks rather than navigation tasks, with no natural distance-to-goal shaping. We instrument them at the full protocol under a task-appropriate subgoal/distance shaping as *measured* non-geometric foils (Section 6.2), confirming that their difficulty is at best weakly reliable and at best weakly predicted by geometry.

Sokoban. The Sokoban box-pushing domain, instantiated on the standard BOXOBAN corpus [?], provides our *mixed* difficulty case: hardness comes partly from spatial layout (corridors, reach, corner deadlocks — geometric) and partly from the box–target assignment and push ordering (combinatorial). BOXOBAN ships three externally curated difficulty tiers, giving an independent signal against which to validate our label. Full domain construction, the push/move DSL, and the dual program-space/state-space solvers are deferred to Section 6.5, where Sokoban serves as the intermediate point ($\rho \approx 0.35$) between the Karel and MiniGrid extremes.

5.2 Environment Sampling and Solver Configuration

For each task we sample a population of grids by varying the generation seed ($\theta = \text{seed}$); the task objective is held fixed and only the layout changes. Each grid is labeled by Eq. (30) with R independent HC search seeds. The HC reference solver uses the configuration of ?: a programmatic search space with mutation scale σ , and Hill Climbing with neighborhood size $k = 250$ and elite count $e = 2$. In Karel we use the programmatic policy space with $\sigma = 0.1$ and a budget of $T = 5000$ program evaluations; in MiniGrid we use the MiniGrid programmatic space with $\sigma = 0.25$ and $T = 2000$. We use $R = 12$ search seeds per grid throughout. The population sizes are 200 grids for TOPOFF, the eight additional Karel tasks, and the two shaped MiniGrid foils (REDBLUEDOOR, PUTNEAR); 80 grids for SNAKE and the earlier-screened HARVESTER/SEEDER; and 60 grids for shaped LAVAGAP.

5.3 Evaluation Protocol

For each task we report three quantities. (1) The label reliability ρ_{rel} of Eq. (31), which gates whether the predictor question is well posed. (2) The best single-descriptor rank correlation, $\max_j |\rho_s(g_j, D_{\text{HC}})|$, as an interpretable lower bound on predictive structure. (3) The multivariate predictive power of the geometric regressor, measured as the Spearman correlation between out-of-fold predictions and the label under 5-fold cross-validation; cross-validation ensures the regressor is scored only on grids it did not train on, so the score reflects generalization rather than memorization. For shaped MiniGrid we additionally instrument three difficulty definitions from the same runs: partial-progress difficulty $1 - \bar{r}_{\text{shaped}}$, success-rate difficulty $1 - \Pr[\text{goal reached}]$, and search-cost difficulty (number of evaluations consumed), to verify the conclusion is not an artifact of one particular difficulty definition.

6 Results

6.1 Karel: Reliable Difficulty that Geometry Fails to Predict

We first confirm the necessity of seed averaging. A single HC run per grid yields a difficulty that is essentially irreproducible: the rank correlation between two independent single-run measurements of the same grid population is $\rho \approx 0.03$. Difficulty defined from one run measures search luck, not the environment. Averaging over $R = 12$ seeds via Eq. (30) restores a reliable label on a subset of tasks (Table 1): TOPOFF and SNAKE reach $\rho_{\text{rel}} = 0.57$ and 0.71 respectively, whereas HARVESTER and SEEDER retain near-zero across-grid variance and reliability and are dropped as uninformative.

On the two headline tasks with a reliable label, the geometric descriptor nonetheless *fails* to predict difficulty: the cross-validated regressor reaches only $\rho = 0.28$ on TOPOFF and $\rho = -0.03$ on SNAKE. This is the central negative result of the Karel domain: on these tasks the difficulty is real and repeatable, but it is governed by the *semantics* of the task — the program logic the search must discover — rather than by the physical layout the descriptor encodes.

Expanded Karel screen. To test whether this geometry-blindness is specific to TOPOFF/SNAKE or a general property of Karel-style difficulty, we ran eight further Karel tasks at the full $N = 200$, $R = 12$ protocol (Table 1). Four fall below the reliability bar that TOPOFF clears at 0.57 — ONESTROKE ($\rho_{\text{rel}} = 0.44$), PATHFOLLOW (0.43), STAIRCLIMBER (0.25), CLEANHOUSE (0.24), FOURCORNERS (0.23) — and are dropped as not well posed, exactly as the reliability gate prescribes. Of the tasks that clear the bar, DOORKEY ($\rho_{\text{rel}} = 0.71$) and MAZE ($\rho_{\text{rel}} = 0.70$) reproduce the TOPOFF/SNAKE pattern exactly: a reliable label that the geometric descriptor cannot predict (CV $\rho = 0.03$ and -0.01), their hardness being sequential program logic (find key, open door, reach target) rather than layout. The exception is WALLAVOIDER, and it is the exception that proves the rule. It carries the most reliable label of any Karel task ($\rho_{\text{rel}} = 0.82$), and there the geometric descriptor predicts difficulty at CV $\rho = 0.42$ — comparable to Sokoban’s mixed regime (Section 6.5) and far above the ≈ 0 of every other reliable Karel task. The reason is that WALLAVOIDER’s hardness is itself spatial: the task requires filling the grid interior with markers while avoiding the ring of cells adjacent to the walls, so the agent’s start position relative to that ring — a geometric quantity, captured here by the start-cell eccentricity ($\rho_s = 0.28$) — directly governs difficulty. WALLAVOIDER thus localizes the geometric/semantic distinction *inside a single domain*: geometry predicts difficulty exactly when difficulty is caused by geometry, whether the domain is MiniGrid, Sokoban, or Karel.

Table 1: Karel at $N = 200$, $R = 12$ (except SNAKE/HARVESTER/SEEDER, $N = 80$). For semantic tasks the difficulty label is reliable but the geometric predictor is blind to it; WALLAVOIDER, whose hardness is spatial, is the lone geometric exception. Reliability ρ_{rel} from Eq. (31); predictive power from 5-fold cross-validated Spearman ρ .

Task	# grids	Label reliability ρ_{rel}	Geometry CV ρ
WALLAVOIDER	200	0.82	0.42
DOORKEY	200	0.71	0.03
SNAKE	80	0.71	-0.03
MAZE	200	0.70	-0.01
TOPOFF	200	0.57	0.28
<i>Below reliability bar ($\rho_{\text{rel}} \leq 0.44$), dropped as not well posed:</i>			
ONESTROKE (0.44), PATHFOLLOW (0.43), STAIRCLIMBER (0.25), CLEANHOUSE (0.24), FOURCORNERS (0.23), HARVESTER (≈ 0.05), SEEDER (≈ 0.00)			

6.2 MiniGrid: Sparse Reward is Degenerate; Shaped Difficulty is Geometric

Under MiniGrid’s native sparse reward, the difficulty label is degenerate: HC terminates almost immediately on every grid and $D_{\text{HC}} \equiv 1$ with zero variance. Increasing the budget from 200 to 1000 to 5000 evaluations does not change this — the landscape is flat, not under-searched — so there is no variance for any predictor to explain. Applying the distance-to-goal shaping of Eq. (33) recovers a graded, reliable label on LAVAGAP, and the geometric descriptor predicts it strongly. Table 2 reports all three difficulty instruments: the conclusion is stable across definitions, with cross-validated ρ between 0.69 and 0.76 and the gap-position features carrying the signal.

We carry only LAVAGAP through the quantitative MiniGrid study as the clean geometric positive control. The domain’s other tasks — REDBLUEDOOR (open the red then blue door in order) and PUTNEAR (place an object beside a target) — are sequencing and manipulation problems with no natural distance-to-goal potential, so the shaping that rescues LAVAGAP does not apply directly; their difficulty is governed by which subgoal to attempt in which order, a *semantic* property. To verify rather than merely assume this, we measured both at the full protocol ($N = 200$, $R = 12$) under a task-appropriate subgoal/distance shaping (a staged BFS-distance potential to the current target door for REDBLUEDOOR, and distance-to-ball then carried-ball-to-target for PUTNEAR). The measurement confirms they belong on the non-geometric side: their labels are only weakly reliable ($\rho_{\text{rel}} = 0.43$ for REDBLUEDOOR and 0.55 for PUTNEAR, at or below the bar TOPOFF clears at 0.57), and even taken at face value the geometric descriptor reaches only CV $\rho = 0.34$ and 0.39 — weak, and far below LAVAGAP’s 0.69–0.76. (A small pilot had suggested REDBLUEDOOR might be substantially more reliable, $\rho_{\text{rel}} \approx 0.9$; that did not survive on the full population, a concrete reminder of why the reliability gate must be computed over the complete grid set rather than a small sample.) These tasks therefore sit with semantic Karel rather than offering an independent geometric test, and we do not promote them to positive controls. The mixed-difficulty regime is instead probed properly by Sokoban (Section 6.5), which admits a principled multi-box progress shaping.

Table 2: Shaped LAVAGAP (60 grids, $R = 12$): once a graded reward yields a reliable label, the geometric descriptor predicts difficulty strongly under all three difficulty definitions.

Difficulty instrument	Label reliability ρ_{rel}	Geometry CV ρ
Partial-progress $1 - \bar{r}_{\text{shaped}}$	0.79	0.76
Success-rate $1 - \Pr[\text{solved}]$	0.74	0.73
Search-cost (evaluations)	0.66	0.69
LAVAGAP, sparse reward (native)	—	degenerate ($\text{Var} = 0$)

6.3 Cross-Domain Synthesis: Reward Structure Gates, Geometry Predicts when Difficulty is Geometric

Read together, the first two domains show that whether the geometric predictor can work on a domain is decided by two conditions applied *in sequence*, like two gates a difficulty signal must pass through. The first gate is the reward: difficulty has to actually *vary* from grid to grid before any predictor has something to explain, and only a graded reward produces that variation — a sparse all-or-nothing reward labels every grid maximally hard and leaves no signal at all. The second gate, reached only once the first is passed, concerns the *nature* of the difficulty: geometry can predict hardness only when that hardness is actually caused by the spatial layout. A domain fails the predictor if it stops at either gate. Native LAVAGAP stops at the first (sparse reward, zero variance, nothing to predict); Karel TOPOFF/SNAKE pass the first but stop at the second (real, reliable difficulty, but the hardness is semantic rather than spatial); shaped LAVAGAP is the only case that clears both.

Table 3 places these first two domains side by side and isolates the governing factors. Two conclusions emerge. First, *reward structure is the first-order gate*: a sparse reward yields a flat search landscape and a difficulty label with no variance, so no descriptor — geometric or otherwise — can predict it; a graded (shaped or naturally dense) reward is a precondition for any difficulty signal to exist. Second, *conditional on a graded reward, the geometric descriptor predicts difficulty only when the difficulty is itself geometric*. In LAVAGAP, hardness is literally the gap’s position and the descriptor succeeds ($\rho \approx 0.7$ –0.76); in Karel TOPOFF/SNAKE, hardness is semantic and the same descriptor is blind ($\rho \approx 0$), despite a reliable label. The expanded Karel screen sharpens this second conclusion into a within-domain statement: WALLAVOIDER, whose Karel difficulty is itself spatial, is read by the same geometric descriptor at $\rho = 0.42$ on an equally reliable label, while the semantic Karel tasks DOORKEY, MAZE, TOPOFF and SNAKE remain blind. Geometric predictability therefore tracks the *nature* of the difficulty, not the identity of the domain.

This pair of poles — a domain where the geometric predictor succeeds and a domain where it reliably fails on an equally real difficulty label — is the empirical core of the paper. It both validates the descriptor on geometric difficulty and

Table 3: Reward structure versus geometric predictability across domains. Difficulty exists only with a graded reward; geometry predicts it only when difficulty is geometric.

Domain / Task	Reward	Difficulty variance	Geometry CV ρ
MiniGrid LAVAGAP	sparse (native)	none (degenerate)	n/a
MiniGrid LAVAGAP	shaped (dense)	real, reliable	0.69–0.76
Karel WALLAVOIDER	dense (native)	real, reliable	0.42
Karel TOPOFF	dense (native)	real, reliable	0.28
Karel DOORKEY	dense (native)	real, reliable	0.03
Karel SNAKE	dense (native)	real, reliable	−0.03

delimits its scope, directly addressing Hypothesis 3: the geometric Lipschitz control holds where difficulty is mediated by environment geometry and breaks where difficulty is mediated by task semantics.

6.4 Cheap Dynamics-Aware Probe: Accessing Semantic Difficulty

The Karel negative result (Section 6.1) is that difficulty there is *semantic*: it is determined by whether the task’s reward is attainable on a given grid, which the static geometric descriptor cannot observe. The cheap probe of Section 4.4 is designed to sense exactly this. Crucially, the probe uses *no* language model and performs *no* search: it executes a fixed library of short hand-specified programs (e.g. “move forward”, “move and place a marker”, “follow a wall while clear”) together with a set of randomly sampled DSL programs, each run once, and records the reward each attains. A short program that exercises the task objective earns reward on an easy grid and none on a hard one, giving a dynamics-aware signal at a fraction of the cost of the full solver.

We evaluate the probe against the *reliable* averaged label (Eq. 30) on the *full* grid populations for which that label is available — 200 TOPOFF and 80 SNAKE grids. (An earlier 50-grid join produced optimistic multivariate scores that did not survive on the full population; we report the full-population result here.) For each task we report, against the same label, the most predictive single feature and the multivariate 5-fold cross-validated regressor, for both the geometric descriptor and the probe (Table 4).

On the full populations the picture is more measured than the small join suggested. The probe is best read at two levels, which here tell different stories. The *single-feature* reading asks whether the one most informative probe number tracks difficulty on its own; the *multivariate* reading asks whether a regressor that combines *all* the probe numbers predicts difficulty out-of-sample. A single probe feature — the best reward reached by any program in the fixed library (`best_probe_reward`) — retains a modest monotone relationship with difficulty ($\rho_s = -0.38$ on TOPOFF, -0.31 on SNAKE), in the expected direction: grids on which a short task-exercising program scores well tend to be easier. This single dynamics-aware feature is stronger than the single best *geometric* feature ($+0.05$ and -0.09 respectively), confirming that the probe reads a semantic signal the static descriptor cannot. At the multivariate level, however, both descriptor families fall to weak cross-validated correlation on the full population (geometry CV $\rho = 0.28 / -0.03$; probe CV $\rho = 0.06 / 0.15$): neither the geometric descriptor nor the probe feature set is a reliable multivariate stand-in for the full label here. The two readings together are the honest result: a single cheap probe number is a useful coarse indicator of the semantic axis geometry misses, but the probe is *not* a drop-in multivariate replacement for the reference solver. (An earlier 50-grid join made the multivariate probe look competitive; that apparent strength was a small-sample artifact and did not survive on the full population.)

Table 4: Cheap probe versus geometric descriptor against the reliable averaged label, full populations ($n = 200$ TOPOFF, $n = 80$ SNAKE). The single best probe feature — the best reward over the fixed library — captures semantic difficulty that the single best geometric feature misses; the multivariate cross-validated scores are weak for both families at full population.

Task	Best single feature (ρ_s)		Regressor (CV ρ)	
	Geometry	Probe	Geometry	Probe
TOPOFF	<code>num_markers</code> : $+0.05$	<code>best_probe</code> : -0.38	0.28	0.06
SNAKE	<code>eccentricity</code> : -0.09	<code>best_probe</code> : -0.31	-0.03	0.15

The negative correlations are in the expected direction: the more reward a cheap task-exercising program attains on a grid, the easier that grid is (lower difficulty). This is the mechanism behind the result — the probe reads the reward landscape directly, whereas the geometric descriptor only sees the layout.

Truncated-search variant. The truncated-search probe (Eq. 32) — a “mini” hill-climbing run, i.e. the full reference solver stopped after only $T_{\text{probe}} \approx 10$ evaluations instead of thousands — replaces the fixed library with $T_{\text{probe}} \approx 10$ perturbations of a *random* base program (no language model), scoring $D_{\text{probe}} = 1 - r_{T_{\text{probe}}}^*$. Unlike the fixed-library probe, it is informative only when the reward is shaped: on shaped LAVAGAP (Section 6.2) it recovers the reference ordering with $\rho_s(D_{\text{probe}}, D_{\text{HC}}) = +0.51$ from roughly ten evaluations, whereas on the sparse Karel rewards it is essentially uninformative ($\rho_s = -0.03$ on TOPOFF, -0.10 on SNAKE). The mechanism is the same sparsity that defeats geometry, and indeed the same first gate of Section 6.3: under a sparse reward a handful of random perturbations almost never produces partial credit, so the truncated search has no gradient to read; a dense shaping potential restores it. The truncated-search probe is thus a useful cheap label *when shaping is available*, complementing the fixed-library probe.

Cost. On these grids the fixed-library probe costs ~ 80 ms and the truncated search ~ 15 ms per grid, versus ~ 330 ms for a full reference-HC run — a $4\text{--}23\times$ saving. The full solver terminates early on these easy grids, so the measured factor is conservative; the gap widens on harder tasks where the reference search runs its full budget. The probe trades a bounded, sub-second rollout cost for a dynamics-aware signal, while the geometric predictor remains the fully execution-free option.

6.5 Third Domain: Sokoban Closes the Geometric–Semantic Spectrum

Karel and MiniGrid stake out two poles — difficulty that is purely semantic (geometry blind, $\rho \approx 0$) and difficulty that is purely geometric (geometry predicts, $\rho \approx 0.7$). A natural worry is that these are hand-picked extremes and that the descriptor framework says nothing about the intermediate regime, where difficulty is *mixed*. We close this gap with a third domain whose difficulty is, by construction, part geometric and part combinatorial: SOKOBAN, the box-pushing puzzle, instantiated on the standard BOXOBAN level corpus [?]. A level is a 10×10 grid on which an agent must push every box onto a target cell; hardness comes both from the spatial layout (corridors, the player’s reach, corner deadlocks — geometric) and from the box–target assignment and push ordering (combinatorial). BOXOBAN additionally ships three externally curated difficulty tiers (UNFILTERED, MEDIUM, HARD), giving an independent difficulty signal against which to validate our label.

Reference solver and label. We label Sokoban with the same DSL+Hill-Climbing reference solver used for Karel and MiniGrid (Section 4.1); the apples-to-apples instantiation and its results are detailed below. Because Sokoban states are also natural game states, we additionally instantiate an equivalent state-space reference — a budgeted stochastic best-first search over board states under the same difficulty-label definition (30) — as an independent cross-check on the program-space label: from random restarts, the search expands states greedily under a shaped reward for a fixed budget of $T = 4000$ expansions, and the per-run difficulty is $1 - (\text{best shaped reward reached})$. The shaping (33) is the multi-box generalization of the distance-to-goal potential: partial credit for the fraction of boxes on target plus normalized progress on the minimum-cost box–target matching distance, with unrecoverable corner-deadlock pushes pruned. As in Karel/MiniGrid both labels average over $R = 12$ search seeds. We sample a stratified pool of 200 levels (70 UNFILTERED, 70 MEDIUM, 60 HARD).

The label is reliable and externally valid. We report the reliability and descriptor analyses on the state-space label (the program-space cross-check is taken up at the end of the section). The averaged label clears the reliability bar decisively, $\rho_{\text{rel}} = 0.985$, and — unlike native-reward MiniGrid — it is graded, not degenerate ($\text{Var}(D) = 0.020$, range $0.0\text{--}0.64$). Crucially, the label is monotone in BOXOBAN’s independent difficulty tiers (mean D : UNFILTERED 0.05, MEDIUM 0.21, HARD 0.25; tier-vs- D Spearman 0.58), confirming that the budgeted-search difficulty measures a real property of the level rather than an artifact of the solver.

Geometry partially predicts; geometry and the probe are complementary. On this reliable label the geometric descriptor lands exactly where the mixed-difficulty hypothesis predicts: between its Karel and MiniGrid extremes. The cross-validated geometric regressor reaches $\rho = 0.35$ — well above the ≈ 0 of semantic Karel but well below the ≈ 0.7 of geometric LAVAGAP (Table 5). This “middle” value is the point of the experiment: a single geometry-predictability number now sits at 0, 0.35, and 0.7 across the three domains, tracing a smooth dial rather than an on/off switch, and 0.35 is what the thesis predicts for a domain whose difficulty is *half* geometric (corridors and deadlocks) and half not (which box goes to which target, in what order). The cheap probe, run as short greedy and random push rollouts, is weaker on

its own here ($\rho = 0.09$): the ordering of the two predictors *flips* relative to Karel, because in Sokoban the spatial layout genuinely carries part of the difficulty, whereas in Karel it carries none. Combining the static descriptor with the cheap probe — simply concatenating the two feature sets and fitting a single cross-validated regressor on the union — yields the best predictor ($\rho = 0.44$). Because the combined score *exceeds* geometry alone (0.35) rather than merely matching it, the probe must be supplying information geometry does not have; the two signals are complementary rather than redundant, each reading a different half of Sokoban’s mixed difficulty.

Table 5: SOKOBAN (200 BOXOBAN levels, $R = 12$): a reliable, externally validated label whose difficulty is mixed. The geometric descriptor predicts partially — between its Karel (≈ 0) and LAVAGAP (≈ 0.7) extremes — and is complementary to the cheap probe.

Predictor	Best single feature (ρ_s)	Regressor (CV ρ)
Geometric descriptor	mean_degree: -0.30	0.35
Cheap probe	probe_best: -0.16	0.09
Geometry + probe	—	0.44
Label reliability ρ_{rel}		0.985
Tier external validity (Spearman)		0.58

Apples-to-apples: the program-space solver reproduces the label. A residual worry is that Sokoban’s tier alignment is an artifact of using a state-space search, whereas Karel and MiniGrid use program-space Hill Climbing. We therefore re-derive the Sokoban label with exactly the Karel/MiniGrid machinery: a small push/move DSL searched by the same Hill-Climbing engine (Section 4.1), under the identical difficulty definition (30) and the same on-target/progress shaping (weights 0.8/0.2), averaged over $R = 12$ seeds on the same 200 levels. The DSL augments the primitive moves with a single `solveStep` macro that plans one box onto its nearest free target (a bounded single-box breadth-first search), so that a finite program can in principle clear a level; this is the one structural difference from the primitive Karel/MiniGrid DSLs and we flag it explicitly. The resulting label is again monotone in the curated tiers (mean D : UNFILTERED 0.09, MEDIUM 0.31, HARD 0.31; tier-vs- D Spearman 0.54, $p < 10^{-15}$) and graded ($\text{Var}(D) = 0.031$; solved fraction 0.64/0.10/0.05 across tiers). Most directly, the two independently constructed solvers agree strongly at the per-level granularity (Spearman $\rho_s = 0.68$, $n = 200$, $p < 10^{-28}$; Table 6, Figure 1), confirming that the difficulty signal is a property of the level and not of the search formulation. The one limitation both solvers share is resolution: each cleanly separates easy from hard but does not distinguish MEDIUM from HARD (program-space 0.31 vs 0.31; state-space 0.21 vs 0.25). This collapse is a property of the measuring instrument, not of the levels. Difficulty is defined as 1—(best reward reached within a *fixed* search budget): an easy level is solved well inside the budget and earns a low score, but once a level is hard enough that the search exhausts its budget without solving, every still-harder level bottoms out at the same “search failed” score regardless of how much harder it truly is. MEDIUM and HARD both sit past this ceiling, so the difference between them is real but lies above the budget’s resolution — the instrument has saturated, much as a scale that maxes out reads the same for two objects both heavier than its limit. A larger budget would in principle pull MEDIUM below HARD and separate them, at proportionally greater cost; we flag this as a shared limitation rather than resolve it here.

Table 6: Two independent reference solvers, one label. The program-space DSL+Hill-Climbing solver used for Karel/MiniGrid reproduces the tier-monotonicity of the state-space search on the same 200 BOXOBAN levels, and the two agree per level at Spearman $\rho_s = 0.68$ ($n = 200$, $p < 10^{-28}$).

Reference solver	mean D by tier			tier-vs- D ρ_s
	unfilt.	medium	hard	
State-space search	0.05	0.21	0.25	0.58
DSL + Hill Climbing	0.09	0.31	0.31	0.54

Sokoban thus completes a graded spectrum across three domains: geometry’s predictive power tracks the degree to which difficulty is mediated by spatial structure — strong in LAVAGAP ($\rho \approx 0.7$), partial in SOKOBAN ($\rho \approx 0.35$), absent in Karel ($\rho \approx 0$) — with the cheap probe supplying the complementary semantic signal where geometry runs out. This is the sharpest available test of Hypothesis 3: the geometric Lipschitz control holds in proportion to how geometric the difficulty actually is.

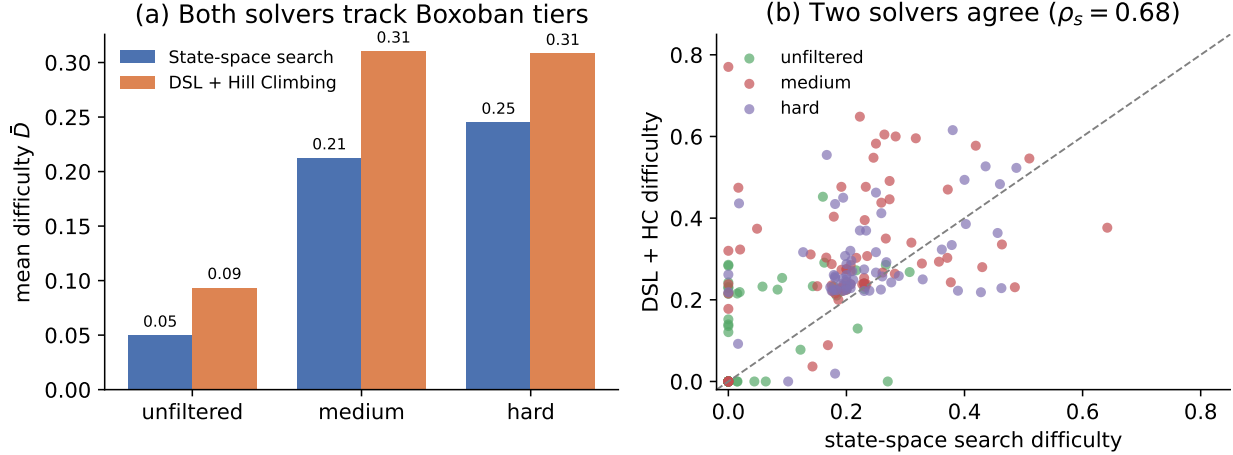


Figure 1: The Sokoban difficulty label is invariant to the search formulation. **(a)** Mean difficulty $\bar{D}$ per BOXOBAN tier for the two reference solvers: both are monotone from UNFILTERED to the curated tiers, and the program-space DSL+HC solver (orange) recovers the same ordering as the state-space search (blue). **(b)** Per-level agreement between the two solvers across all 200 levels (Spearman $\rho_s = 0.68$); colors denote the curated tier and the dashed line is $y = x$.

6.6 Downstream Use Case: Predictor-Curated Training Sets

The predictor’s practical value is as a *prefilter* that identifies which candidate environments are worth the expense of policy training (Section 3.5). A large pool of procedurally generated tasks is typically redundant: many share the same difficulty and descriptor-space location and contribute little once one representative is trained on. Because the predictor scores each environment from cheap descriptors, it can rank a pool by predicted difficulty and by coverage (Eq. 14 and the ridge-leverage score of Section 3.4), keep a small representative subset, and discard the redundant remainder before any expensive training. We test this use case in shaped LAVAGAP. The choice of domain is forced rather than arbitrary: coverage-based curation can only help where the descriptor space it covers actually tracks difficulty, which is the case only in LAVAGAP ($\rho \approx 0.7$). In Karel the geometric descriptor is blind to difficulty ($\rho \approx 0$), so spreading a subset out in descriptor space would be spreading it along axes uncorrelated with hardness; in Sokoban the descriptor captures only about half of difficulty ($\rho \approx 0.35$). LAVAGAP is therefore the one domain in which the prefilter’s curation step is meaningful, and we use it as the controlled setting for the use-case test.

Protocol. We generate a pool of $N = 150$ LAVAGAP layouts and score each with the cheap signals of Section 4 (geometry descriptor plus a 10-evaluation mini-HC probe), with no full solve. “Training a reference agent on a set S ” is instantiated faithfully to our difficulty definition: we run Hill Climbing over the same DSL program space (Section 4.1) to find the single program that maximizes *mean* shaped reward across all layouts in S , and we charge the training cost in environment-evaluations (program evaluations $\times |S|$). We compare three training sets: the **full** pool ($N = 150$); a **curated** subset of size k , selected by restricting to a target-difficulty band (the $[0.15, 0.85]$ quantile range of the probe) and then greedily covering standardized descriptor space by farthest-point (k -center) selection; and a **random** subset of the same size k , the ablation that isolates whether the predictor — rather than merely training on fewer tasks — is responsible for any effect. Each trained program is evaluated on a held-out set of $M = 80$ fresh layouts (disjoint seeds). Because this single-program learner makes partial progress on held-out layouts rather than fully solving them, the reported metric is mean held-out shaped reward. To make the curated-versus-random comparison statistically fair, we draw an *ensemble* of 6 curated subsets (each from an 85% pool subsample with a randomized k -center start) and 6 random subsets, and train each with 6 search seeds, giving 36 runs per condition; the full pool is trained with 6 seeds. We sweep $k \in \{10, 15, 20, 30, 50\}$ (k/N from 7% to 33%).

The prefilter saves compute (claim 3). Table 7 shows the headline result clearly: at every subset size, training on $k \ll N$ layouts reaches the same held-out shaped reward as training on the full pool. The full pool attains 0.404 ± 0.076 ; every curated and random subset, down to $k = 10$ (7% of the pool), lands within this band while consuming $2.7\times$ to $14.9\times$ fewer environment-evaluations. This is a direct, positive validation of empirical claim (3) of Section 3.5: a large procedurally generated pool is highly redundant, and a small representative subset suffices to match full-pool generalization at a fraction of the training cost. It also depends on claim (2): the prefilter is only useful because the cheap descriptors rank and cover the pool well enough that a 7% slice is representative.

Table 7: Shaped LAVAGAP prefilter ($N = 150$ pool, $M = 80$ held-out, 36 runs per subset condition). Held-out mean shaped reward (mean $\pm$ s.d.) for the full pool, a predictor-curated size- k subset, and a random size- k subset; Δ is curated – random; p is a one-sided Mann–Whitney test of curated $>$ random; “cost” is the full-pool training cost divided by the curated cost (environment-evaluation ratio). *The small subset matches full-pool generalization at 2.7–15 $\times$ less compute at every k , but the predictor-curated subset is not statistically distinguishable from a random subset of the same size in this domain.*

k (k/N)	Full	Curated- k	Random- k	Δ (cur–rnd)	p / cost
10 (7%)	0.404 ± 0.076	0.378 ± 0.106	0.370 ± 0.082	+0.008	0.47 / 14.9 $\times$
15 (10%)	0.404 ± 0.076	0.390 ± 0.100	0.370 ± 0.072	+0.020	0.23 / 9.9 $\times$
20 (13%)	0.404 ± 0.076	0.382 ± 0.083	0.409 ± 0.126	−0.027	0.70 / 7.6 $\times$
30 (20%)	0.404 ± 0.076	0.406 ± 0.101	0.390 ± 0.137	+0.015	0.08 / 5.1 $\times$
50 (33%)	0.404 ± 0.076	0.397 ± 0.107	0.407 ± 0.118	−0.010	0.61 / 2.7 $\times$

The predictor does not beat random selection in this domain. The ablation is, however, a negative result, and we report it as such. Across the entire k -sweep the curated subset is not statistically distinguishable from a random subset of the same size: the curated–random difference is small and changes sign (−0.027 to +0.020), and no k reaches significance (smallest $p = 0.08$ at $k = 30$). An earlier single-curated-subset comparison appeared to show a large advantage at $k = 15$; that effect did not survive the fair ensemble ablation and was an artifact of comparing one fortunate curated draw against a distribution of random draws. We therefore cannot claim that descriptor-coverage curation outperforms uniform sampling here. The most plausible explanation is that the LAVAGAP descriptor space is low-dimensional and the pool is dense within it, so a uniform random subset already covers the space about as well as an explicit k -center selection; the predictor’s curation has little redundancy left to exploit beyond what random subsampling captures. We expect the curation advantage to be more pronounced in higher-dimensional descriptor spaces, or in pools containing many degenerate or unsolvable tasks on which random sampling would waste budget — but that is a conjecture this experiment does not establish. The compute-saving claim stands on its own and does not depend on it: in this domain the saving comes from pool redundancy, which the prefilter exploits with or without descriptor-aware curation.

6.7 Downstream Use Case II: Cheap Geometric Difficulty Builds a Neural Curriculum

The prefilter of Section 6.6 exploits one face of a difficulty estimate—which environments are worth training on—in a regime where every task is individually learnable and a large pool is merely redundant. We now test the complementary face: using the estimate to *order* tasks from easy to hard for a learner that genuinely benefits from a curriculum. This shifts the setting in two deliberate ways. First, the learner is a neural policy trained with PPO rather than a program recovered by Hill Climbing, because curriculum ordering is a property of *gradient-based* learning—the training signal concentrates on the tasks the agent is on the verge of solving, a mechanism program-space search does not possess. Second, the regime is *hard-exploration*: the hardest tasks lie beyond the agent’s exploration horizon, so a flat schedule over the full task distribution fails outright and ordering—not curation—becomes the operative lever. Both choices make this a genuine test of whether the cheap geometric difficulty signal validated above is not merely *correlated* with hardness but *actionable*.

Domain and regime. We use MiniGrid LAVACROSSING, in which one or more lava rivers, each with a single safe crossing, separate the agent from the goal; stepping into lava is fatal, so reward is sparse and exploration is the bottleneck. We instantiate a 3×3 difficulty grid crossing three grid sizes $\{9, 13, 17\}$ with three river counts $\{1, 2, 3\}$, spanning an easy corner (small grid, one river; learnable from scratch) to a hard corner (large grid, three rivers; never solved from scratch within budget). From this grid we enumerate 90 training layouts and 36 disjoint held-out layouts (separate generation seeds), each presented as the standard $7 \times 7 \times 3$ egocentric observation with the action set restricted to the three navigation primitives. Held-out success—the fraction of held-out layouts solved—is the sole outcome metric, so a method that merely overfits the training layouts gains nothing.

Curriculum machinery. All conditions share one curriculum controller and differ *only* in the difficulty signal that orders the tasks. The training pool is partitioned into three difficulty bins (terciles of the signal). Training begins on the easy bin alone; the next bin unlocks when the policy clears a competence gate (held success ≥ 0.55 on a probe set) or when a deadline elapses (a bin is force-unlocked after it has been active for a fraction 0.25 of the budget). The deadline guarantees that every method eventually trains on the *full* distribution, so the comparison isolates the *order* in which tasks are introduced rather than which tasks are seen—a fair ordering test. Each run trains for 1.5M environment steps,

evaluating held-out success at 30 checkpoints, and we report the mean over 10 seeds with a 95% normal-approximation confidence interval.

Conditions. Seven difficulty signals drive the controller. RANDOM orders tasks arbitrarily and, with all bins unlocked from the start, is the flat-schedule floor. ORACLE is the expensive empirical reference: it trains a reference PPO policy and labels difficulty by its measured failure rate—the neural analogue of the D_{HC} label of Section 4.1, and the only condition that is *not* oracle-free. GEOMETRY is the cheap, oracle-free signal at the heart of this paper: a closed-form score read directly from the layout (river count, grid size, and induced detour length), evaluated in milliseconds with no rollouts and no label fitting. PROBE scores each layout by a handful of short random-walk rollouts. COMBINED averages the geometry and probe ranks. ADAPTIVE-COARSE and ADAPTIVE-FINE start from the geometry order but re-score the remaining tasks online using the *current* policy’s success rate—coarsely (only when a bin unlocks, ~ 2 re-scores) or finely (at every checkpoint, 30 re-scores).

Cheap geometry matches the expensive oracle. Table 8 and Figure 2 report the headline result. The flat RANDOM schedule never succeeds—0/10 seeds reach the target and final held-out success is 0.02—confirming the hard-exploration regime in which curriculum ordering is the difference between learning and not learning. Against this floor, the cheap oracle-free GEOMETRY signal reaches a final held-out success of 0.50 ± 0.21 , statistically indistinguishable from the expensive ORACLE’s 0.48 ± 0.20 (overlapping confidence intervals); the cheap online ADAPTIVE-COARSE is the most reliable of all, solving the target on 8/10 seeds with the tightest interval (0.54 ± 0.16). We state the finding as a *match*, not a win: with 10 seeds the differences among ORACLE, GEOMETRY, and ADAPTIVE-COARSE are well within noise. The scientific content is precisely that a difficulty signal costing milliseconds attains the same curriculum quality as one costing a full reference-policy training run. The weaker signals locate themselves sensibly: PROBE alone is insufficient (3/10) and COMBINED recovers most of the geometry signal (6/10). Figure 3 shows the held-out learning curves—RANDOM pinned to the floor while the geometry- and oracle-driven curricula climb together—and Figure 4 shows *where* the gain materializes: RANDOM fails in every (size $\times$ river) cell, whereas the curriculum methods crack the large, multi-river grids that are unreachable from scratch.

Table 8: Neural curriculum in hard-exploration LAVACROSSING (90 train / 36 held-out layouts; 1.5M steps; 10 seeds; all conditions oracle-free except ORACLE). “Reached” counts seeds crossing the held-out target (0.30) at any checkpoint; † steps-to-target is censored at the 1.5M budget when no seed reaches it; “speedup” is the RANDOM steps-to-target over the condition’s. *The cheap oracle-free GEOMETRY and ADAPTIVE-COARSE match the expensive ORACLE within overlapping intervals, while the flat RANDOM schedule never succeeds.*

Condition	Reached	Final success	Steps-to-target ($\times 10^3$)	Speedup
RANDOM (flat)	0/10	0.02 ± 0.03	1500 †	1.00 $\times$
ORACLE (expensive)	7/10	0.48 ± 0.20	840 ± 346	1.79 $\times$
GEOMETRY (cheap)	7/10	0.50 ± 0.21	815 ± 316	1.84 $\times$
PROBE	3/10	0.22 ± 0.20	1330 ± 225	1.13 $\times$
COMBINED	6/10	0.40 ± 0.19	955 ± 302	1.57 $\times$
ADAPTIVE-FINE	7/10	0.33 ± 0.20	880 ± 289	1.70 $\times$
ADAPTIVE-COARSE	8/10	0.54 ± 0.16	825 ± 303	1.82 $\times$

Cost: the oracle’s hidden budget. Because all conditions train the same 1.5M curriculum steps, their cost difference lies entirely in producing the difficulty signal (Table 9). GEOMETRY and the adaptive variants pay essentially nothing: the geometric score is closed-form, and online re-scoring reuses evaluations the controller already performs (ADAPTIVE-COARSE adds under 3% wall time). ORACLE alone must train a full reference policy *before* the curriculum can begin; on identical hardware this pre-curriculum build takes 541 s for a single reference seed (and ~ 1620 s at the default of three), so it—not the curriculum—dominates the oracle’s wall-clock budget. GEOMETRY and ADAPTIVE-COARSE thus reach oracle-level held-out success at about three-quarters of the oracle’s total wall (1492 s vs. $1469+541$ s), or under half at the default of three reference seeds. The lone cautionary note is ADAPTIVE-FINE: re-scoring at every checkpoint inflates wall time by $\sim 74\%$ (2603 vs. 1492 s) yet *lowers* final success relative to ADAPTIVE-COARSE—more difficulty re-estimation is not better.

Robustness and reproducibility. Figure 5 plots all ten per-seed outcomes per condition: the GEOMETRY result is broad (7/10 seeds end above target, median 0.66) rather than a single fortunate seed, and its median exceeds the ORACLE median—the means are pulled down by occasional collapse-to-zero seeds common to all conditions in this regime, not propped up by lucky highs. Table 10 lists the full configuration. One reproducibility caveat: aggregate

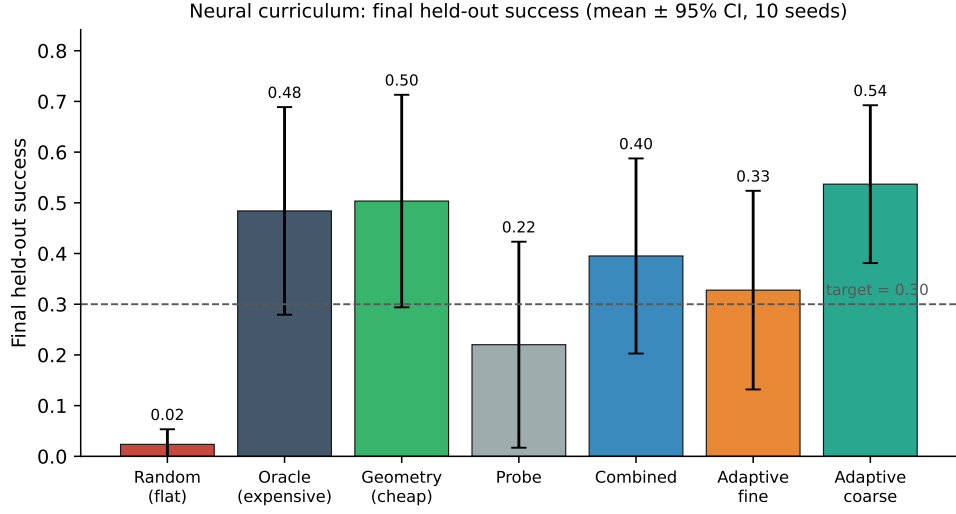


Figure 2: Final held-out success of the neural LAVACROSSING curriculum by difficulty signal (mean $\pm$ 95% CI over 10 seeds; dashed line is the target). The flat RANDOM schedule sits on the floor, while the cheap oracle-free GEOMETRY and ADAPTIVE-COARSE signals match the expensive ORACLE within overlapping intervals.

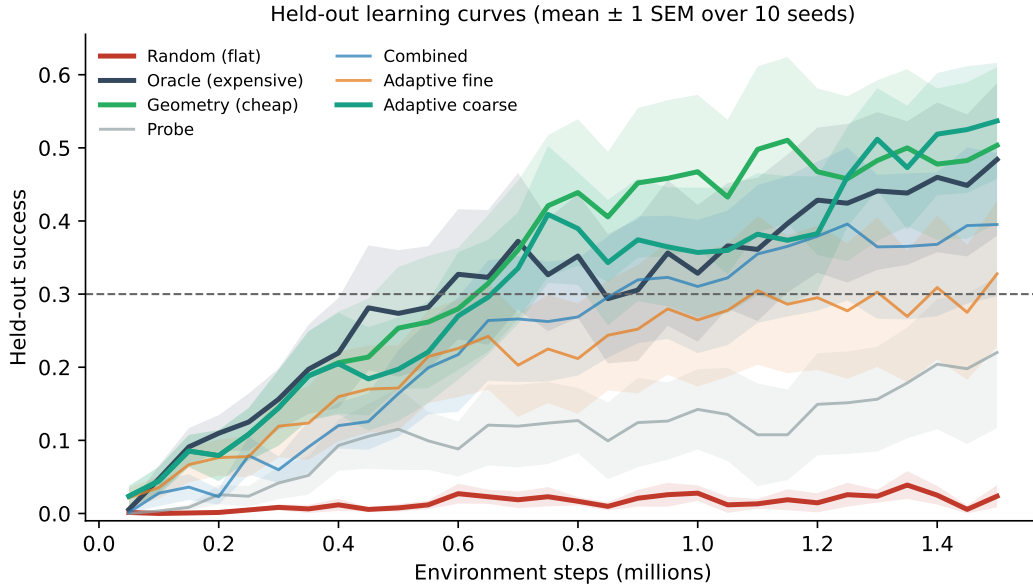


Figure 3: Held-out learning curves (mean $\pm$ 1 SEM over 10 seeds). RANDOM never leaves the floor, while the geometry-, oracle-, and adaptive-coarse-driven curricula climb together; ADAPTIVE-FINE lags despite costing the most.

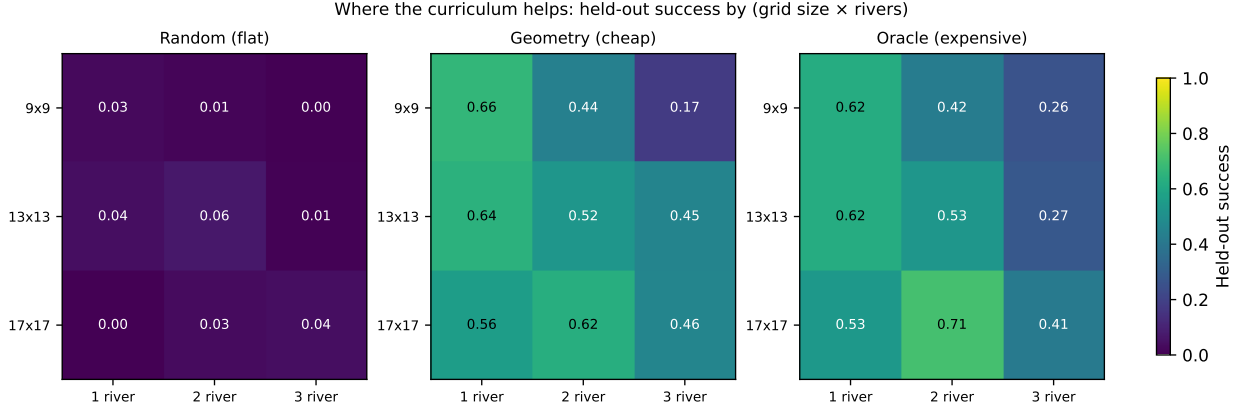


Figure 4: Where the curriculum helps: mean held-out success per (grid size $\times$ lava rivers) cell at the end of training. RANDOM (left) fails in every cell; the cheap GEOMETRY curriculum (middle) and the expensive ORACLE (right) both crack the large, multi-river grids that are unreachable from scratch.

Table 9: Wall-clock cost (mean seconds over 10 seeds, identical CPU hardware). All conditions train the same 1.5M curriculum steps (rightmost column) and differ only in how the difficulty signal is produced. Only ORACLE pays a pre-curriculum cost—it must train a reference policy before ordering can begin, and the default of three reference seeds triples it. *The cheap signals add essentially nothing, so GEOMETRY and ADAPTIVE-COARSE reach oracle-level held-out success (Table 8) at a fraction of the total cost.*

Method	Difficulty signal	Pre-curriculum build (s)	Curriculum wall (s)
GEOMETRY	closed-form score	0	1492
ADAPTIVE-COARSE	geometry + online re-score ($\times 2$)	0	1529
PROBE	~ 720 random rollouts	0	1552
COMBINED	geometry + probe	0	1561
ADAPTIVE-FINE	geometry + online re-score ($\times 30$)	0	2603
ORACLE	reference-policy failure rate	541	1469

statistics reproduce across platforms, but because the layout-sampling RNG draws differ between CPU architectures, individual per-seed values are not bit-identical across machines.

Table 10: Configuration for the neural LAVACROSSING curriculum experiment (Section 6.7).

Component	Setting
Learner	PPO (Stable-Baselines3), small CNN policy, CPU
Budget	1.5M env steps; held-out eval at 30 checkpoints
Task pool	LAVACROSSING, sizes $\{9, 13, 17\} \times$ rivers $\{1, 2, 3\}$ 90 training / 36 held-out layouts (disjoint seeds)
Observation / actions	$7 \times 7 \times 3$ egocentric; 3 navigation actions
Curriculum	3 difficulty bins; competence gate 0.55; deadline 0.25 of budget
Target	held-out success ≥ 0.30
Oracle label	reference PPO, 6×10^5 steps, 1 reference seed, 12 eval episodes
Seeds	0–9 (10 per condition)

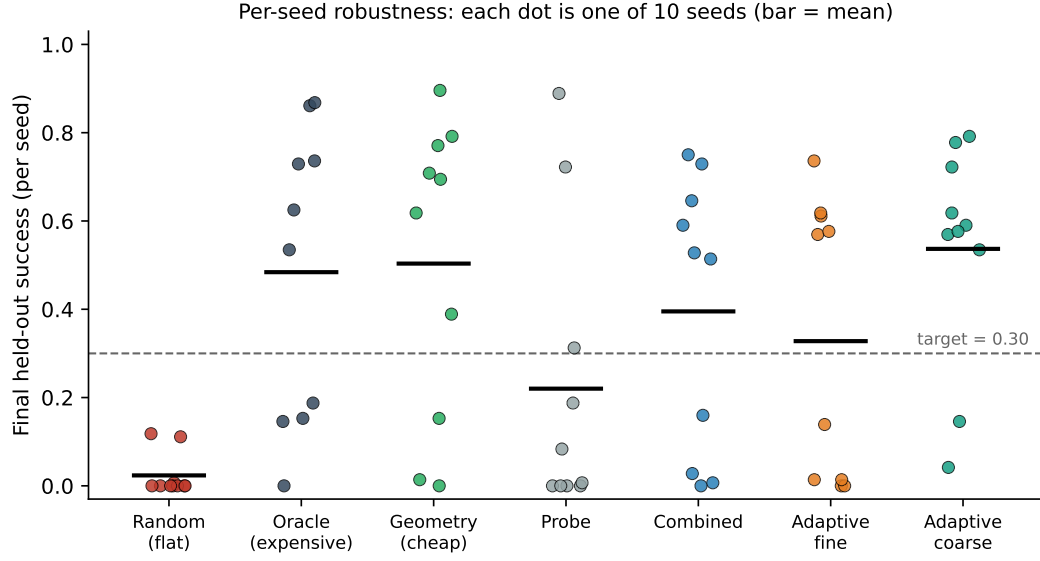


Figure 5: Per-seed robustness: each dot is one of 10 seeds’ final held-out success and the bar is the condition mean. The GEOMETRY result is broad (7/10 seeds above target, median 0.66) rather than a single lucky seed, and its median exceeds the ORACLE median.

References